

รายงาน

เรื่อง ระบบ POS และบริหารสต็อกร้านกาแฟหลายสาขา

(Whatever Cafe POS & Smart Inventory System)

โดย

นายเทิดไท ศรีสวัสดิ์	รหัสนิสิต 67160010
นายปัญญวัฒน์ ประเสริฐพงษ์	รหัสนิสิต 67160034
นายสมพงษ์ เหลืองอ่อน	รหัสนิสิต 67160044
นายดนัยวัฒน์ เพ็ชรหนองชุม	รหัสนิสิต 67160184
นายวัชรพงษ์ วงษ์สุวรรณ์	รหัสนิสิต 67160198

เสนอ

อาจารย์วิทวัส พันธุมจินดา

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 88734065 System Analysis and Designs

คณะวิทยาการสารสนเทศ สาขาวิชาเทคโนโลยีสารสนเทศเพื่ออุตสาหกรรมดิจิทัล

ภาคเรียนที่ 1 ปีการศึกษา 2569

มหาวิทยาลัยบูรพา

สารบัญ

1. บทนำ 3
2. การวิเคราะห์ความเป็นไปได้ 5
3. การวิเคราะห์ความต้องการ 7
4. User Stories และ Use Case 9
5. การออกแบบเชิงวัตถุ (OOAD) 11
6. การออกแบบฐานข้อมูล 14
7. การออกแบบกระบวนการและพฤติกรรมระบบ 16
8. การออกแบบ Input/Output และ UI 19
9. สถาปัตยกรรมระบบ 23
10. ผลการทดสอบ Usability 24
11. ผลการรวมระบบ (Integration) 26
12. ผลการตรวจสอบคุณภาพและความปลอดภัย 27
13. สรุปผลและข้อเสนอแนะ 29
14. ภาคผนวก 31

1. บทนำ

1.1 บริบทของร้านกาแฟ

โครงงานนี้พัฒนาระบบ POS และบริหารสต็อกสำหรับร้านกาแฟที่มีหลายสาขา ในกรณีศึกษา พนักงานหน้าร้านจดออเดอร์ด้วยมือลงกระดาษแล้วส่งต่อให้บาริสต้าด้วยการเดินไปส่งหรือตะโกนบอก การสรุปยอดขายและการนับสต็อกก็ทำด้วยมือเช่นกัน

1.2 ปัญหาที่พบ

จากการสัมภาษณ์เจ้าของร้านจำลอง (wk03) ปัญหาสำคัญคือ

- จดออเดอร์ผิดบ่อยในช่วงเร่งด่วน (07:00 ถึง 09:00 น.) เช่น ลืมท็อปปิ้งหรือสลับระดับความหวาน และกระดาษออเดอร์สูญหาย
- บาริสต้าอ่านลายมือไม่ออกและทำเครื่องดื่มสลับคิว
- สต็อกไม่ถูกตัดอัตโนมัติ ผู้บริหารไม่เห็นยอดขายรวมของทุกสาขาแบบทันที
- ระบบสมาชิกเดิมเป็นบัตรปั๊มกระดาษที่ลูกค้าทำหายบ่อย

1.3 วัตถุประสงค์ของโครงงาน

1. ลดเวลาการสั่งและคิดเงินหน้าร้านให้เสร็จภายใน 60 วินาทีต่อออเดอร์
2. ลดความผิดพลาดของตัวเลือกเครื่องดื่มโดยส่งออเดอร์ตรงถึงหน้าจอบาริสต้า (เป้าหมายไม่เกิน 2 วินาที)
3. ตัดสต็อกอัตโนมัติและแจ้งเตือนเมื่อสต็อกต่ำกว่า 10 หน่วย
4. ให้ผู้จัดการวิเคราะห์ยอดขายและเมนูขายดีได้

1.4 ทีมงานและแนวทางพัฒนา

ทีมมีสมาชิก 5 คน (ตามตารางในหน้าปก) เลือกพัฒนาแบบ Iterative Model (wk01) เพราะโครงงานมีเวลาจำกัดหนึ่งภาคการศึกษา นำข้อเสนอแนะมาปรับในรอบถัดไปได้ และลดความเสี่ยงโดยทดสอบเป็นรอบ องค์ประกอบของระบบสารสนเทศที่ระบุไว้ใน wk01 มี 6 ส่วน คือ บุคลากร กระบวนการ ข้อมูล ฮาร์ดแวร์ ซอฟต์แวร์ และเครือข่าย

1.5 เทคโนโลยีที่ใช้

ส่วน	เทคโนโลยี
Backend	Node.js และ Express
ฐานข้อมูล	MySQL 8 (รันผ่าน Docker Compose)

ส่วน	เทคโนโลยี
Frontend	HTML, CSS และ JavaScript แบบ vanilla (หน้าเว็บสัมผัสสำหรับแท็บเล็ตและเดสก์ท็อป)
การยืนยันตัวตน	token ที่ลงลายเซ็นด้วย HMAC และการควบคุมสิทธิ์ตามบทบาท (RBAC)

2. การวิเคราะห์ความเป็นไปได้

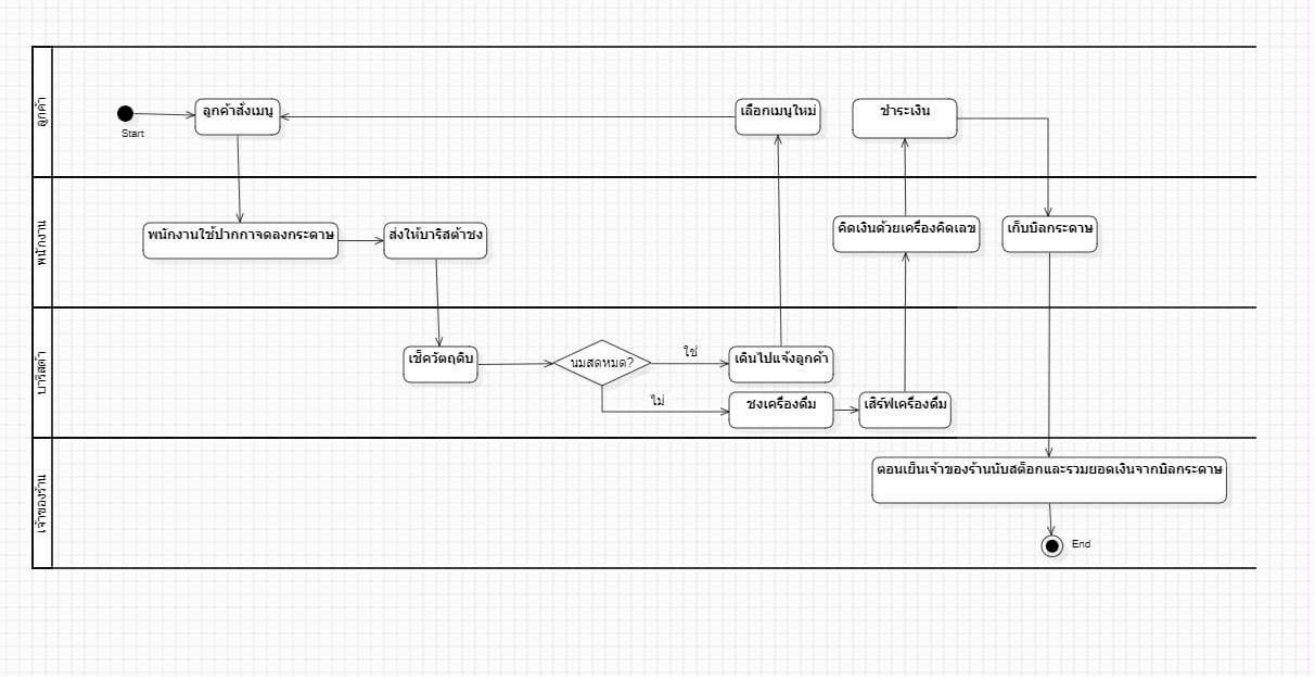
รายงานฉบับเต็มอยู่ที่ wk02-feasibility-report.md

ด้าน	ข้อสรุป
เทคนิค (Technical)	ใช้ Node.js, Express และ MySQL ซึ่งทีมมีความรู้ระดับปานกลาง ความเสี่ยงหลักคือการเชื่อมต่อ QR Payment Gateway จริง ซึ่งทีมแก้โดยศึกษาเอกสารก่อนและจำลองการชำระเงินแทนการเชื่อมต่อจริง
เศรษฐศาสตร์ (Economic)	ต้นทุนโดยประมาณ 30,000 บาท บวกค่าเช่าเซิร์ฟเวอร์และค่าบำรุงรักษา คาดว่าคุ้มทุนภายในประมาณ 6 เดือน
กฎหมาย (Legal)	ระบบเก็บข้อมูลส่วนบุคคลของสมาชิก (ชื่อ เบอร์โทร แต้ม) และพนักงาน จึงเกี่ยวข้องกับ PDPA (มาตรา 19, 22, 23, 33, 37) ภาษีมูลค่าเพิ่มของใบเสร็จ (ประมวลรัษฎากร มาตรา 86/4 และ 86/6) และสัญญาอนุญาตซอฟต์แวร์ ไม่เป็นอุปสรรคต่อการพัฒนา แต่ต้องมีการขอความยินยอมสมาชิกและกำหนดระยะเวลาเก็บข้อมูลก่อนใช้งานจริง ซึ่งยังไม่ได้ทำในต้นแบบ แหล่งอ้างอิงอยู่ในรายงานฉบับเต็ม wk02-feasibility-report.md
การปฏิบัติงาน (Operational)	พนักงานต้องเปลี่ยนจากจดกระดาษเป็นบันทึกผ่านหน้าจอ ทีมออกแบบ UI ให้ใช้ง่าย และวางแผนอบรมแผนกละ 2 ถึง 3 ชั่วโมง
เวลา (Schedule)	สัปดาห์ 1 ถึง 4 วิเคราะห์และออกแบบ สัปดาห์ 5 ถึง 12 พัฒนา สัปดาห์ 13 ถึง 16 ทดสอบและนำเสนอ

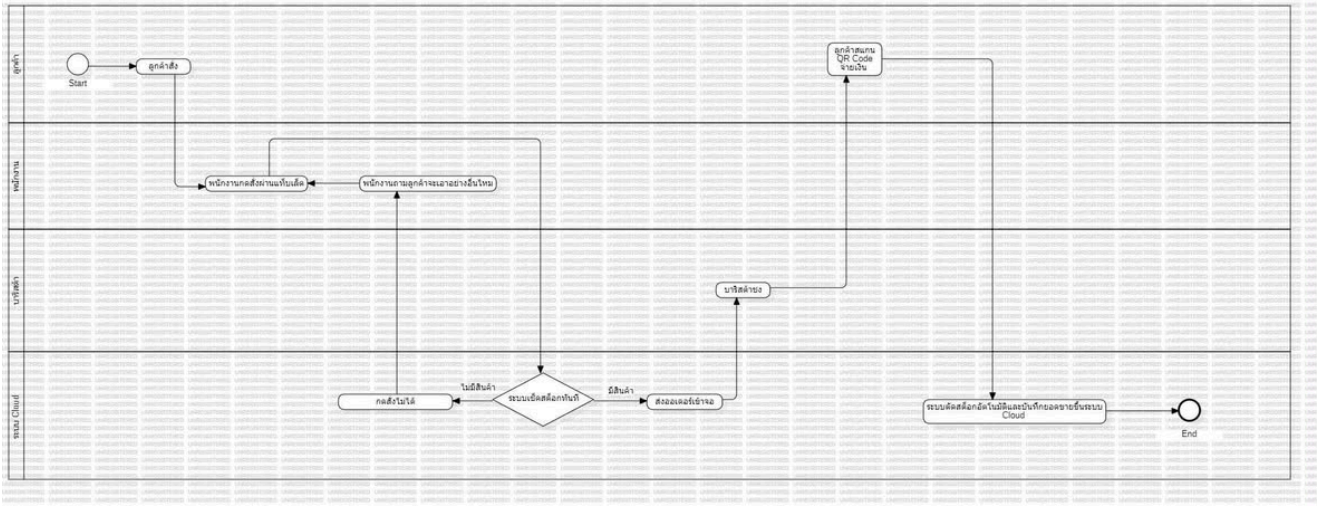
ข้อสรุป: ควรดำเนินโครงการต่อ เพราะผลประโยชน์คุ้มค่ากับต้นทุนและทีมมีความพร้อมด้านเทคนิคและเวลา

2.1 BPMN กระบวนการเดิมและกระบวนการที่ต้องการ

กระบวนการเดิม (As-Is)



กระบวนการใหม่ด้วยระบบ POS (To-Be)



3. การวิเคราะห์ความต้องการ

รายละเอียดฉบับเต็มอยู่ที่ wk03-requirements.md

3.1 การสัมภาษณ์ผู้ใช้

สมาชิกทีมสวมบทบาทเจ้าของร้าน (นายสมพงษ์ เหลืองอ่อน) และอีกสี่คนเป็นทีมวิเคราะห์ระบบ ใช้คำถาม 8 ข้อ แบ่งเป็นปัญหาปัจจุบัน (2 ข้อ) ความคาดหวัง (3 ข้อ) และข้อจำกัด (3 ข้อ) ผลสรุป

หัวข้อ	สิ่งที่ได้จากการสัมภาษณ์
ปัญหาปัจจุบัน	จดออเดอร์ผิด ส่งคิวให้บาริสต้าด้วยกระดาษทำให้สลับคิว
ความคาดหวังด้านสต็อก	ตัดสต็อกอัตโนมัติตามสาขาเมื่อคิดเงินเสร็จ และแจ้งเตือนเมื่อใกล้หมด
ความคาดหวังด้านรายงาน	แดชบอร์ดยอดขายรวม จำนวนบิล ยอดเฉลี่ยต่อบิล สัดส่วนรายได้ตามสาขา และ 10 อันดับเมนูขายดี
สมาชิก	ใช้เบอร์โทร 10 หลักรับส่วนลด 5% และสะสมแต้ม
งบประมาณ	ไม่เกิน 150,000 บาท
เวลา	พร้อมใช้งานภายใน 3 เดือน เพื่อเปิดสาขาใหม่
เทคโนโลยี	เว็บแอปแบบสัมผัสที่พนักงานใหม่เรียนรู้ได้ภายใน 10 นาที

3.2 ผู้ใช้งานระบบ

ระบบมี 4 บทบาทตามคอลัมน์ role ของตาราง user คือ พนักงานหน้าร้าน (Cashier) บาริสต้า (Barista) ผู้จัดการร้าน (Manager) และผู้ดูแลระบบส่วนกลาง (Admin) ส่วนลูกค้าเป็นผู้มีส่วนได้ส่วนเสียภายนอกที่ไม่ได้ล็อกอินเข้าระบบ

3.3 ความต้องการเชิงหน้าที่ (FR)

รหัส	ความต้องการ
FR-01	หน้าจอ POS เลือกเมนู ค้นหา กรองหมวดหมู่ และปรับแต่งขนาดแก้ว ความหวาน ท็อปปิ้ง
FR-02	คำนวณยอดรวม ส่วนลดสมาชิก 5% ภาษีมูลค่าเพิ่ม 7% รับชำระเงินสดหรือ QR PromptPay และออกใบเสร็จ
FR-03	ส่งออเดอร์ที่ชำระแล้วเข้าหน้าจอคิวของบาริสต้า (KDS)
FR-04	บาริสต้าอัปเดตสถานะคิว pending, preparing, ready, completed
FR-05	ตรวจและตัดสต็อกอัตโนมัติ ปฏิเสธเมื่อสต็อกไม่พอ และแจ้งเตือนเมื่อสต็อกต่ำกว่า 10
FR-06	ค้นหาและสมัครสมาชิก Cafe Club ด้วยเบอร์โทร 10 หลัก
FR-07	รายงานยอดขายตามช่วงวันที่และสาขาสำหรับผู้จัดการ
FR-08	ล็อกอินและควบคุมสิทธิ์ตามบทบาทและสาขา

รหัส	ความต้องการ
FR-09	จัดการเมนูและสต็อก (เพิ่ม แก้ไข ลบ)
FR-10	ศูนย์ควบคุมส่วนกลางสำหรับ Admin จัดการพนักงาน สมาชิก และสาขา
FR-11	แจ้งเตือน "เมนูนี้สต็อกหมด" เมื่อแตะเมนูที่หมด (เพิ่มจากผลทดสอบ usability)
FR-12	ยืนยันซ้ำก่อนบันทึกการชำระเงิน QR (เพิ่มจากผลทดสอบ usability)

FR-11 และ FR-12 บันทึกไว้ในประวัติการเปลี่ยนแปลงความต้องการของ wk03 (วันที่พบ 17 ตุลาคม 2569)

3.4 ความต้องการไม่เชิงหน้าที่ (NFR) และข้อจำกัด

รหัส	หัวข้อ	สาระสำคัญ
NFR-01	ประสิทธิภาพ	ออเดอร์จากแคชเชียร์ปรากฏบนหน้าจอบาริสต้าไม่เกิน 2 วินาที
NFR-02	การใช้งาน	ออกแบบสำหรับจอสัมผัส ปุ่มกดมีขนาดเหมาะสม
NFR-03	ความน่าเชื่อถือ	พร้อมให้บริการไม่น้อยกว่า 99.5% และข้อมูลสอดคล้องกัน
NFR-04	การรองรับอุปกรณ์	แสดงผลถูกต้องบนแท็บเล็ตและเดสก์ท็อป
NFR-05	ความปลอดภัย	ปกป้องรหัสผ่านและตรวจ token และบทบาททุกครั้ง

ข้อจำกัด: เวลาไม่เกิน 3 เดือน งบประมาณไม่เกิน 150,000 บาท และต้องเป็น Web Application ด้วย Node.js, Express, JavaScript และ MySQL

3.5 ความต้องการฉบับรวม (SRS)

SRS ฉบับสมบูรณ์รวมความต้องการจาก wk03 และ wk04 ข้อมูลจาก wk07 และการออกแบบ input/output จาก wk10 เก็บเป็นไฟล์แยกที่ wk10-srs.md ไม่ซ้ำในเล่มนี้

---

4. User Stories และ Use Case

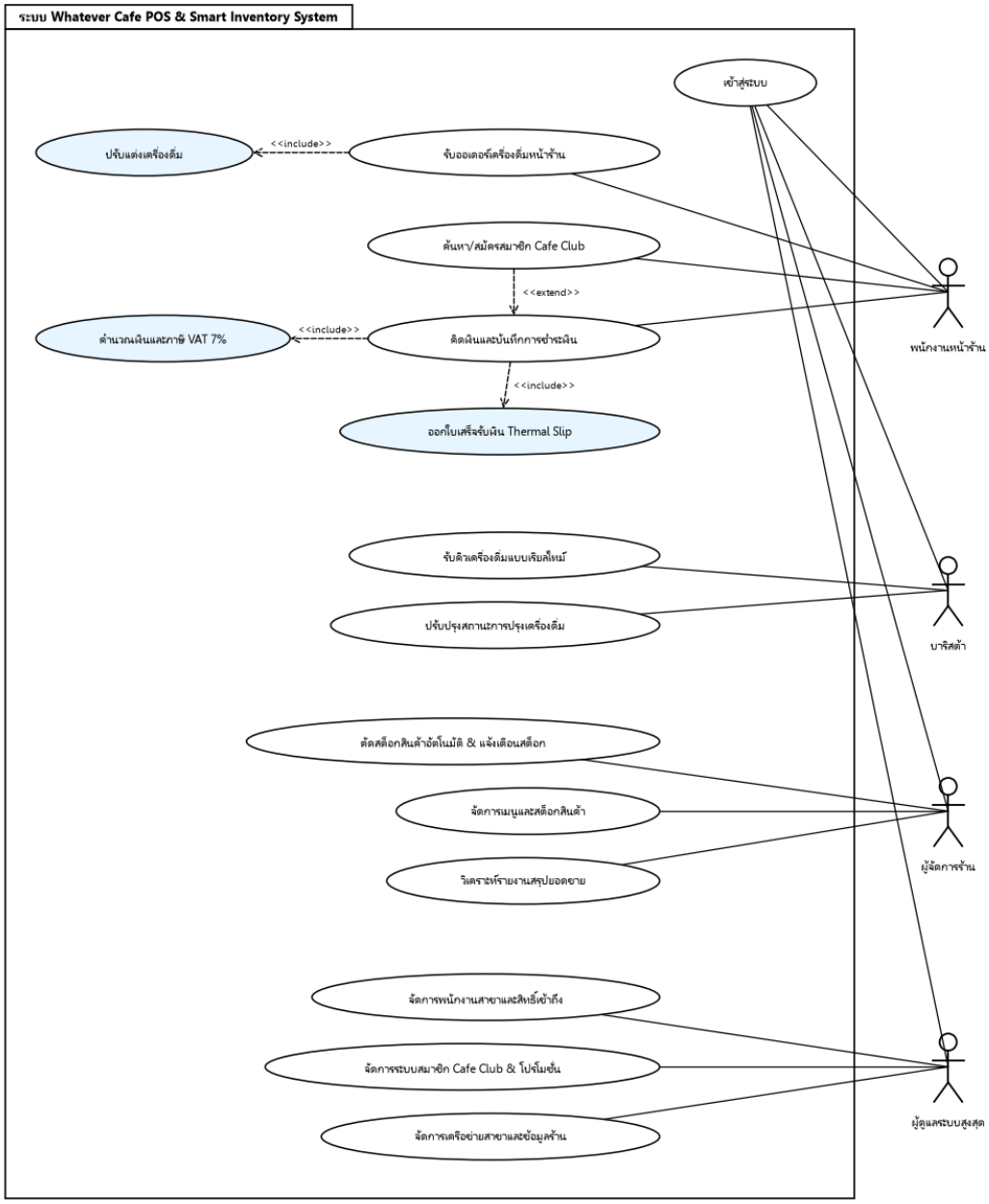
รายละเอียดฉบับเต็มอยู่ที่ wk04-user-stories.md

4.1 User Stories

ทีมเขียน User Story 10 เรื่อง แต่ละเรื่องมี Acceptance Criteria แบบ Given-When-Then และตรวจตามหลัก INVEST

US	เรื่อง	ผู้ใช้
US-01	เลือกเมนูและปรับแต่งเครื่องดื่ม	Cashier
US-02	คิดเงิน ชำระเงิน และออกใบเสร็จ	Cashier
US-03	รับออเดอร์ในบาร์แบบเรียลไทม์	Barista
US-04	จัดการสถานะคิวเครื่องดื่ม	Barista
US-05	ตัดสต็อกอัตโนมัติและแจ้งเตือน	ระบบ
US-06	ค้นหาและสมัครสมาชิก Cafe Club	Cashier
US-07	ดูรายงานสรุปยอดขาย	Manager
US-08	ยืนยันตัวตนและควบคุมสิทธิ์ตามบทบาท	ทุกบทบาท
US-09	จัดการเมนูและสต็อก	Manager
US-10	ศูนย์ควบคุมส่วนกลาง	Admin

4.2 Use Case Diagram

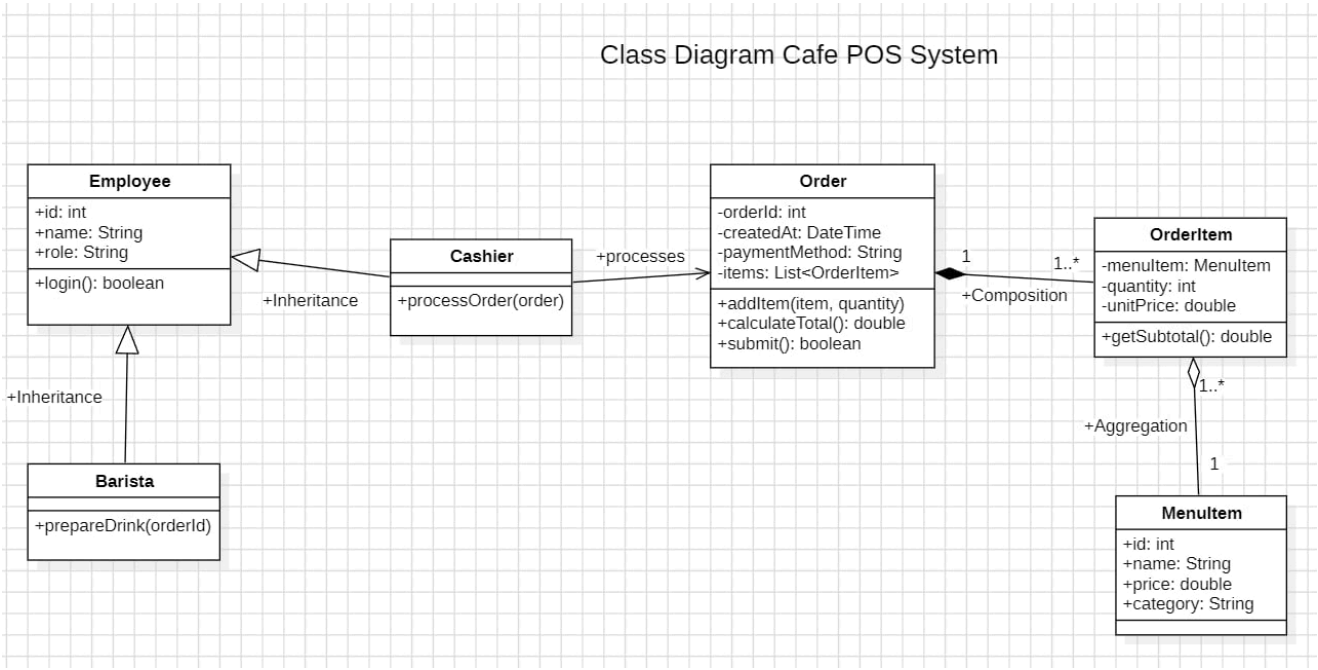


4.3 Use Case Description

มีคำอธิบาย Use Case ที่สำคัญ 5 เรื่อง ได้แก่ UC-01 รับออเดอร์ ปรับแต่งเครื่องดื่ม และชำระเงินหน้าร้าน (ชื่อคงที่ตลอดตามที่วิชากำหนดเพราะเป็นแกนของ Coding Sprint), UC-02 รับคิวและเตรียมเครื่องดื่ม, UC-03 จัดการเมนูและสต็อก, UC-04 วิเคราะห์รายงานยอดขาย และ UC-05 ศูนย์ควบคุมส่วนกลาง UC-01 และ UC-02 ระบุ actor เงื่อนไขก่อนและหลัง main flow และ alternative flow ไว้ครบ (เช่น สต็อกไม่พอ UC-01 ข้อ 2a) ส่วน UC-03 ถึง UC-05 ระบุ Use Case ID, actor, คำอธิบาย และ main flow

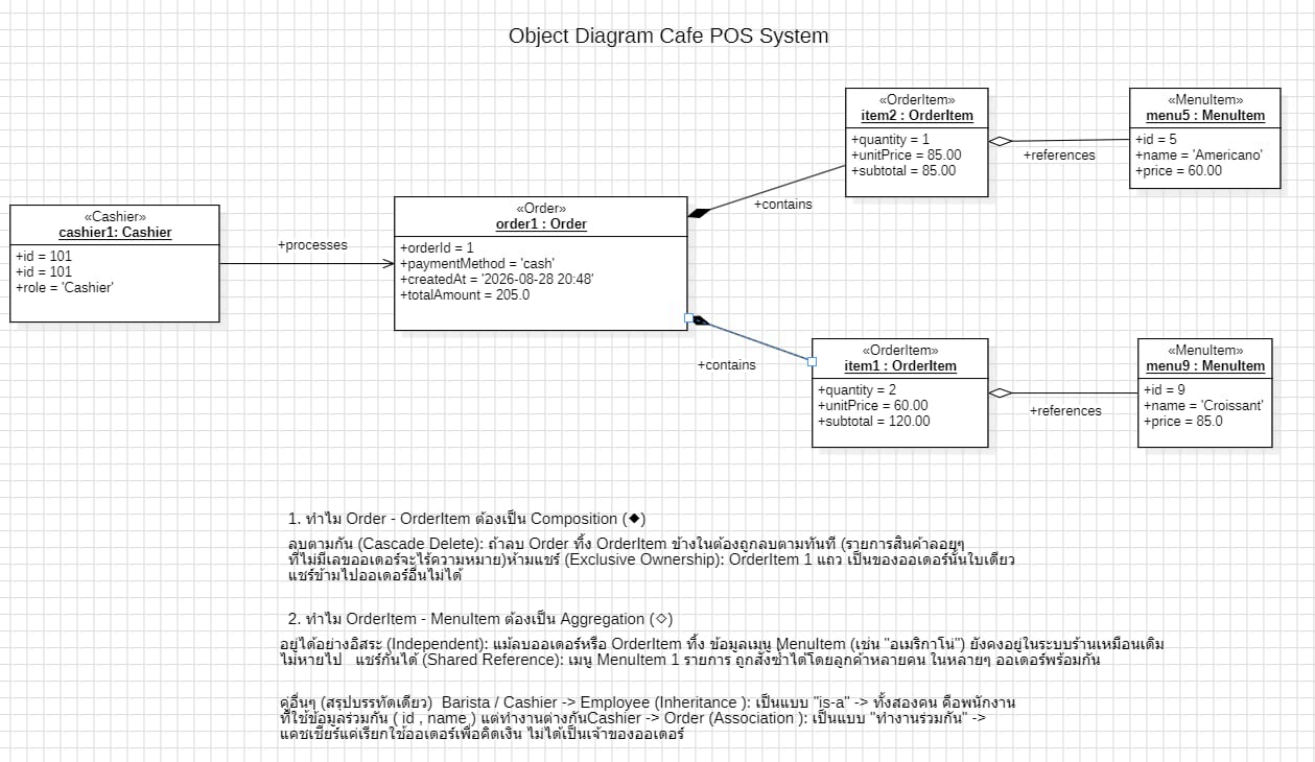
5. การออกแบบเชิงวัตถุ (OOAD)

5.1 Class Diagram



Class Diagram ที่ส่งใน wk06 มีคลาส Employee (และคลาสลูก Cashier กับ Barista ผ่าน Inheritance), Order, OrderItem และ MenuItem โดย Order กับ OrderItem เป็น Composition (ลบออเดอร์แล้วรายการสินค้าถูกลบตาม เพราะรายการสินค้าไม่มีความหมายเมื่อไม่มีออเดอร์ และ OrderItem หนึ่งแถวเป็นของออเดอร์ใบเดียว) ส่วน OrderItem กับ MenuItem เป็น Aggregation (เมนูยังอยู่ในระบบแม้ออเดอร์หรือรายการสินค้าถูกลบ และเมนูหนึ่งถูกอ้างอิงได้จากหลายออเดอร์) และ Cashier เชื่อมกับ Order แบบ Association

5.2 Object Diagram



Object Diagram แสดงตัวอย่างออเดอร์หนึ่งใบที่มีรายการสินค้า 2 รายการ พร้อมค่าของ attribute และเหตุผลของการเลือกชนิดความสัมพันธ์

5.3 ความต่างระหว่าง diagram กับโค้ดปัจจุบัน

ตามแนวทางของวิชา เมื่อโค้ดที่ implement จริงต่างจาก diagram ของสัปดาห์ก่อน ไม่ต้องย้อนแก้ไฟล์เดิม ให้บันทึกความต่างและเหตุผลไว้ ตารางนี้สรุปความต่างของ Class Diagram ฉบับ wk06 กับระบบปัจจุบัน

ใน Class Diagram (wk06)	ในโค้ดและฐานข้อมูลปัจจุบัน	เหตุผลที่ต่าง
Employee, Cashier, Barista (Inheritance)	ตาราง user คอลัมน์เดียว role (admin, manager, cashier, barista) ตรวจสิทธิ์ด้วย requireRole	ระบบต้องมีบัญชีล็อกอินและ 4 บทบาท ซึ่งต่างกันที่สิทธิ์ ไม่ใช่โครงสร้างข้อมูล จึงใช้ค่า role แทนคลาสย่อย
บทบาท admin และ manager ไม่มีใน diagram	มีในระบบ (FR-08, FR-09, FR-10)	diagram ใน wk06 วาดเฉพาะบทบาทที่เกี่ยวกับการสั่งซื้อ ไม่ได้ครอบคลุม admin และ manager ที่ระบุไว้ใน wk03 และ wk04 แล้ว

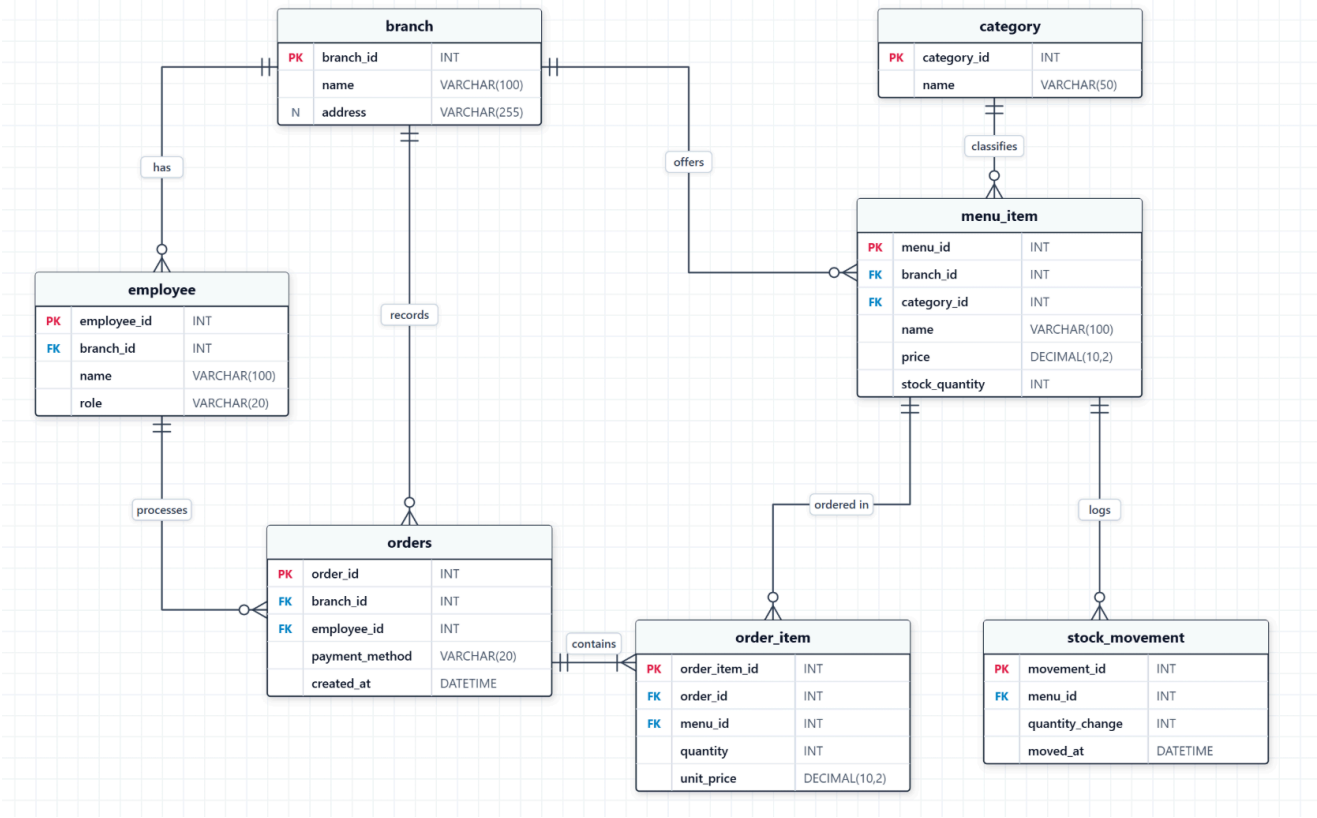
ใน Class Diagram (wk06)	ในโค้ดและฐานข้อมูลปัจจุบัน	เหตุผลที่ต่าง
คลาส Order และ OrderItem พร้อม addItem, calculateTotal, getSubtotal, submit	ใน wk06 ทีม implement จริงที่ src/models/Order.js (commit a51ca50, 28 สิงหาคม 2569) ปัจจุบันไม่มีไฟล์นี้ (ถูกลบใน commit 7b4521e, 17 ตุลาคม 2569) การคำนวณยอดรวมอยู่ใน orderController.js	เมื่อเข้า Sprint 2 ทีมใช้ controller และ orderModel.js ที่เขียน SQL โดยตรง ทำให้คลาสนี้ไม่ได้ถูกเรียกใช้ จึงลบออกตอนจัดระเบียบโปรเจกต์
Order มี calculateTotal() และ Object Diagram มี totalAmount	ไม่มีคอลัมน์ total_amount ใน schema ยอดรวม ส่วนลด และ VAT คำนวณฝั่ง server เมื่อสร้างออเดอร์ และฝั่งหน้าจอเมื่อแสดงใบเสร็จ	ออกแบบให้ยอดมาจากรายการสินค้า (unit_price และ quantity) เพื่อไม่ให้ข้อมูลซ้ำกัน (สอดคล้องกับ schema ใน wk07)
Employee.login()	authController.login ตรวจรหัสผ่านแบบ hash (scrypt) พร้อมจำกัดจำนวนครั้งที่ลอง	เพิ่มจากการตรวจความปลอดภัยใน wk14

diagram ฉบับนี้ยังคงเป็นฉบับที่ส่งใน wk06 ทีมไม่ได้วาดใหม่

---

6. การออกแบบฐานข้อมูล

6.1 ER Diagram (เวอร์ชันที่ส่งใน wk07)



ER Diagram ที่ส่งใน wk07 มี 7 ตาราง คือ branch, employee, category, menu_item, orders, order_item, stock_movement และไฟล์ SQL คือ wk07-schema.sql

6.2 โครงสร้างฐานข้อมูลในระบบจริง

ระบบจริงใช้ไฟล์ schema.sql ที่ root ของ repository มี 8 ตาราง

ตาราง	หน้าที่	คอลัมน์หลัก
category	หมวดหมู่เมนู	category_id, name
branch	สาขา	branch_id, name, address
user	พนักงานและผู้ดูแลระบบ	user_id, branch_id, name, email (ไม่ซ้ำ), password (hash), role
menu_item	เมนูของแต่ละสาขา	menu_id, branch_id, category_id, name, price, stock_quantity
member	สมาชิก Cafe Club	member_id, name, phone (ไม่ซ้ำ), points, discount_rate, joined_at
orders	หัวบิล	order_id, branch_id, user_id, member_id, payment_method, status, created_at

ตาราง	หน้าที่	คอลัมน์หลัก
order_item	รายการสินค้าในบิล	order_item_id, order_id, menu_id, quantity, unit_price, notes
stock_movement	ประวัติการเปลี่ยนสต็อก	movement_id, menu_id, quantity_change, moved_at

6.3 ความต่างระหว่าง ER Diagram เวอร์ชันแรกกับ schema จริง

ตามแนวทางของวิชา ทีมไม่ได้ย้อนแก้ ER Diagram และ wk07-schema.sql ที่ส่งไปแล้ว ความต่างและเหตุผลมีดังนี้

รายการ	ER Diagram (wk07)	ระบบจริง	เหตุผลที่ต่าง
ตารางพนักงาน	employee	user พร้อม email, password, role	เพิ่มระบบล็อกอินและบทบาทใน Coding Sprint 2 (FR-08)
สมาชิก	ไม่มี	ตาราง member และ orders.member_id	ER Diagram ใน wk07 ไม่ได้รวมตารางสมาชิกที่ระบุไว้ใน FR-06 (wk03) จึงเพิ่มตอน implement
สถานะคิว	ไม่มี	orders.status	เพิ่มคิวบาริสต้าตาม FR-03 และ FR-04
หมายเหตุเครื่องดื่ม	ไม่มี	order_item.notes	เก็บขนาด ความหวาน ท็อปปิ้งที่ลูกค้าเลือก

ผู้ตรวจควรใช้ตารางในหัวข้อ 6.2 และไฟล์ schema.sql เป็นหลักเมื่อต้องการดูโครงสร้างจริง

---

7. การออกแบบกระบวนการและพฤติกรรมระบบ

7.1 Sequence Diagram ของการสั่งซื้อ

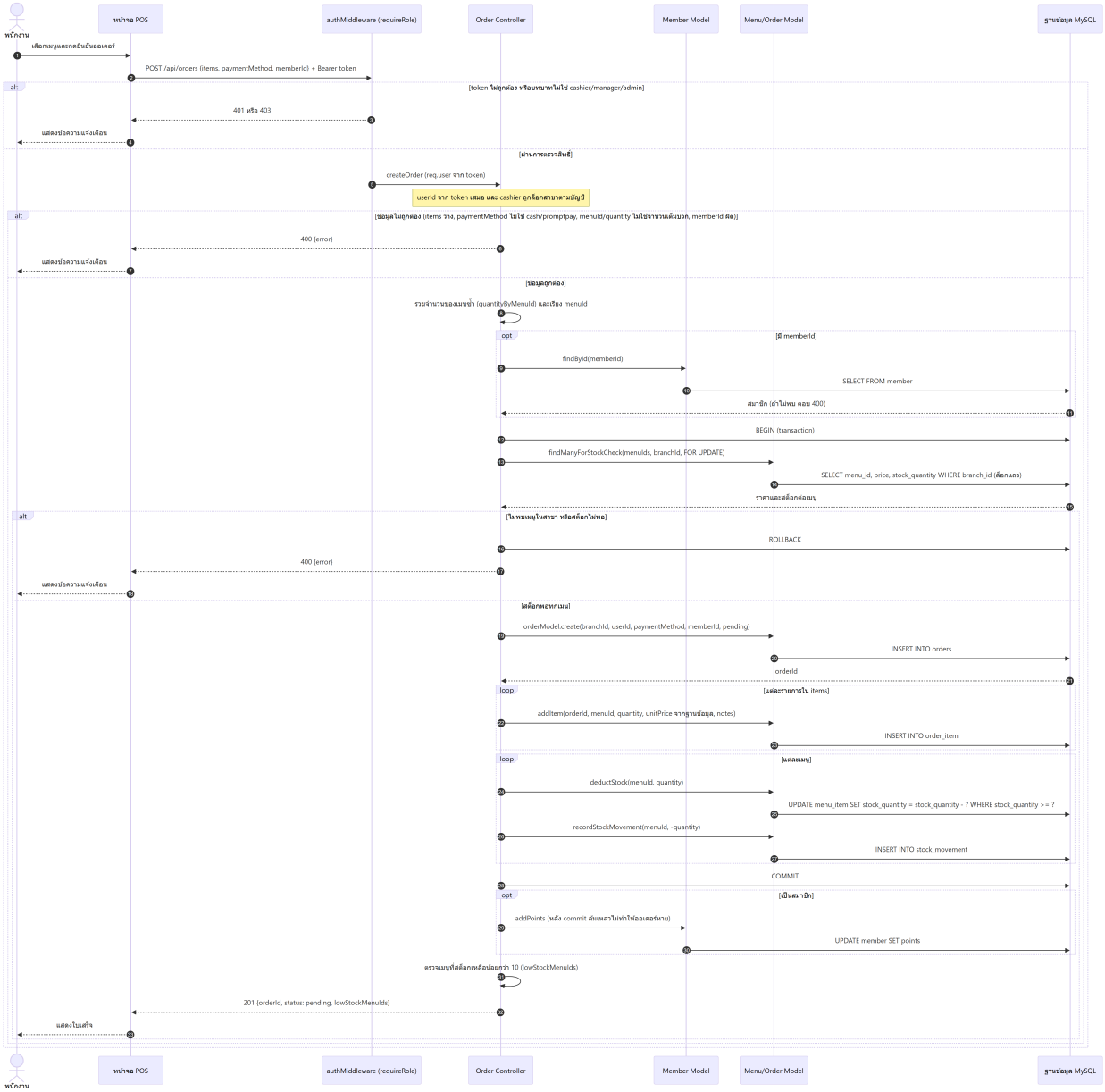


diagram เวอร์ชันปรับปรุงหลังแก้ไขใน wk14 (แทนที่เวอร์ชัน wk09 เดิม เพื่อให้ตรงกับโค้ดปัจจุบันและแก้ปัญหาการซ้อน alt ที่ไม่ถูกต้องในเวอร์ชันแรก) แสดงลำดับของ POST /api/orders ตั้งแต่การตรวจสิทธิ์ (role cashier/manager/admin) ผ่าน token การตรวจข้อมูลนำเข้า การตรวจสมาชิก (ถ้ามี memberId) การเริ่ม transaction การรวมจำนวนเมนูที่ซ้ำและตรวจสต็อกแบบล็อกแถว (FOR UPDATE) การสร้างออเดอร์ การเพิ่มรายการ การตัดสต็อกพร้อมบันทึก stock_movement การ commit การสะสมแต้มสมาชิกหลัง commit ไปจนถึงการตอบ 201 พร้อมรายการเมนูที่สต็อกต่ำ โดยมีกรณีทางเลือก (alt) ที่ซ้อนกันตามลำดับจริง คือ ไม่ผ่านสิทธิ์ (401/403), ข้อมูลไม่ถูกต้อง (400), และไม่พบเมนู/สต็อกไม่พอ (400 พร้อม rollback)

7.2 Business Logic ที่ใช้จริง

กฎธุรกิจหลักของ createOrder

- ราคาคำนวณจากฐานข้อมูลฝั่ง server เสมอ ไม่รับราคาจาก client
- ผู้สั่งและสาขาใช้ค่าจาก token (cashier ถูกล็อกสาขาตามบัญชี) ไม่เชื่อค่าที่ client ส่งมา
- เมนูต้องเป็นของสาขาเดียวกับผู้สั่ง สต็อกต้องพอสำหรับจำนวนรวมของเมนูที่ซ้ำกัน
- วิธีชำระเงินต้องเป็น cash หรือ promptpay
- ส่วนลดสมาชิก 5% และ VAT 7% คำนวณตามนโยบายที่ตกลงกัน
- ขั้นตอนบันทึกทั้งหมดอยู่ใน transaction เดียว ล้มกลางทางจะ rollback
- สถานะออเดอร์เปลี่ยนตามลำดับ pending, preparing, ready, completed เท่านั้น

7.3 ความต่างระหว่าง diagram กับโค้ดปัจจุบัน

diagram ในหัวข้อ 7.1 เป็นเวอร์ชันที่วาดใหม่หลังการแก้ไขใน wk14 เพื่อให้ตรงกับโค้ดปัจจุบันและแก้ปัญหาการซ้อน alt ของเวอร์ชันแรกที่ส่งใน wk09 (เวอร์ชัน wk09 เดิมแยก alt การตรวจสต็อกออกมาเป็นกล่องแยกในระดับเดียวกับ alt ตรวจข้อมูล ทำให้ดูเหมือน flow ไหลต่อได้แม้ข้อมูลไม่ถูกต้อง) เทียบกับ diagram wk09 ฉบับที่ส่งจริง โค้ดของ createOrder เปลี่ยนไปในประเด็นต่อไปนี้

ประเด็น	ใน Sequence Diagram ฉบับส่งจริง (wk09)	ในโค้ดปัจจุบัน (สะท้อนใน diagram ที่ปรับปรุงแล้ว)	เหตุผลที่ต่าง
ขอบเขตธุรกรรม	คำสั่งแยกกัน	อยู่ใน transaction เดียว (BEGIN ถึง COMMIT) ล้มกลางทางจะ rollback	แก้ความเสี่ยง R3 ใน wk14 ป้องกันออเดอร์ที่ตัดสต็อกไม่ครบ
การตรวจและตัดสต็อก	SELECT แล้ว UPDATE แยกกัน	ล็อกแถวด้วย SELECT ... FOR UPDATE และตัดแบบมีเงื่อนไข stock_quantity >= ?	แก้ race condition เมื่อสั่งพร้อมกัน
ผู้สั่งและสาขา	ส่ง employeeId, branchId ใน body	ใช้ค่าจาก token	แก้ความเสี่ยงด้านสิทธิ์ (R1) ไม่เชื่อค่าจาก client
ประวัติสต็อก	ไม่มี	บันทึกลง stock_movement ในธุรกรรมเดียวกัน	FR-05 กำหนดให้บันทึก แต่ไม่ได้ implement ใน Sprint 2 (R13)
การตรวจสิทธิ์	ไม่มี	ตรวจ token และบทบาทก่อนเข้า controller	แก้ R1 และ R2
สะสมแต้มสมาชิก	ไม่ได้วาด	ทำหลัง commit เพื่อไม่ให้ความล้มเหลวของการให้แต้มทำให้ออเดอร์หาย	diagram wk09 วาดเฉพาะ flow หลักของการสั่งซื้อ ไม่ได้รวมการสะสมแต้ม (FR-06)

ประเด็น	ใน Sequence Diagram ฉบับส่งจริง (wk09)	ในโค้ดปัจจุบัน (สะท้อนใน diagram ที่ปรับปรุงแล้ว)	เหตุผลที่ต่าง
โครงสร้าง alt (สต็อก)	แยกเป็น alt อิสระหลัง alt ตรวจข้อมูล	ซ้อนอยู่ใน branch "ข้อมูลถูกต้อง" ของ alt ตรวจข้อมูล	เวอร์ชัน wk09 วาดโครงสร้างคลาดเคลื่อนจากตรรกะจริง จึงปรับปรุงให้ซ้อนถูกต้องตามลำดับการทำงานจริง

8. การออกแบบ Input/Output และ UI

เอกสารต้นฉบับ: wk10-input-output-design.docx และ wk10-wireframe.docx

8.1 ฟอร์มรับออเดอร์ (POS Input)

ออกแบบแต่ละช่องกรอกให้มีชนิดช่อง การตรวจสอบ ค่าเริ่มต้น และการตอบสนองทันที

ช่อง	ชนิด	การตรวจสอบ
สาขา	Dropdown	ต้องเลือกสาขา ค่าเริ่มต้นคือสาขาปัจจุบัน
เมนูเครื่องดื่ม	ปุ่ม/การ์ด	ต้องเลือกเมนูที่มีในระบบ
ขนาดแก้ว	ปุ่ม	S, M, L
ท็อปปิ้ง	ปุ่ม/Checkbox	เลือกได้เฉพาะที่ระบบกำหนด
ความหวาน	ปุ่ม	0%, 25%, 50%, 100%
จำนวน	ตัวเลข/ปุ่ม	จำนวนเต็มมากกว่า 0 ค่าเริ่มต้น 1
เบอร์สมาชิก	ช่องข้อความ	ตัวเลข 10 หลัก แสดงข้อมูลและแต้มเมื่อพบ
วิธีชำระเงิน	Radio/ปุ่ม	ต้องเลือกเงินสดหรือ QR
ยืนยันการชำระเงิน	ปุ่ม	ต้องมีรายการและวิธีชำระเงิน แสดงสถานะสำเร็จและใบเสร็จ

8.2 ใบเสร็จและรายงาน

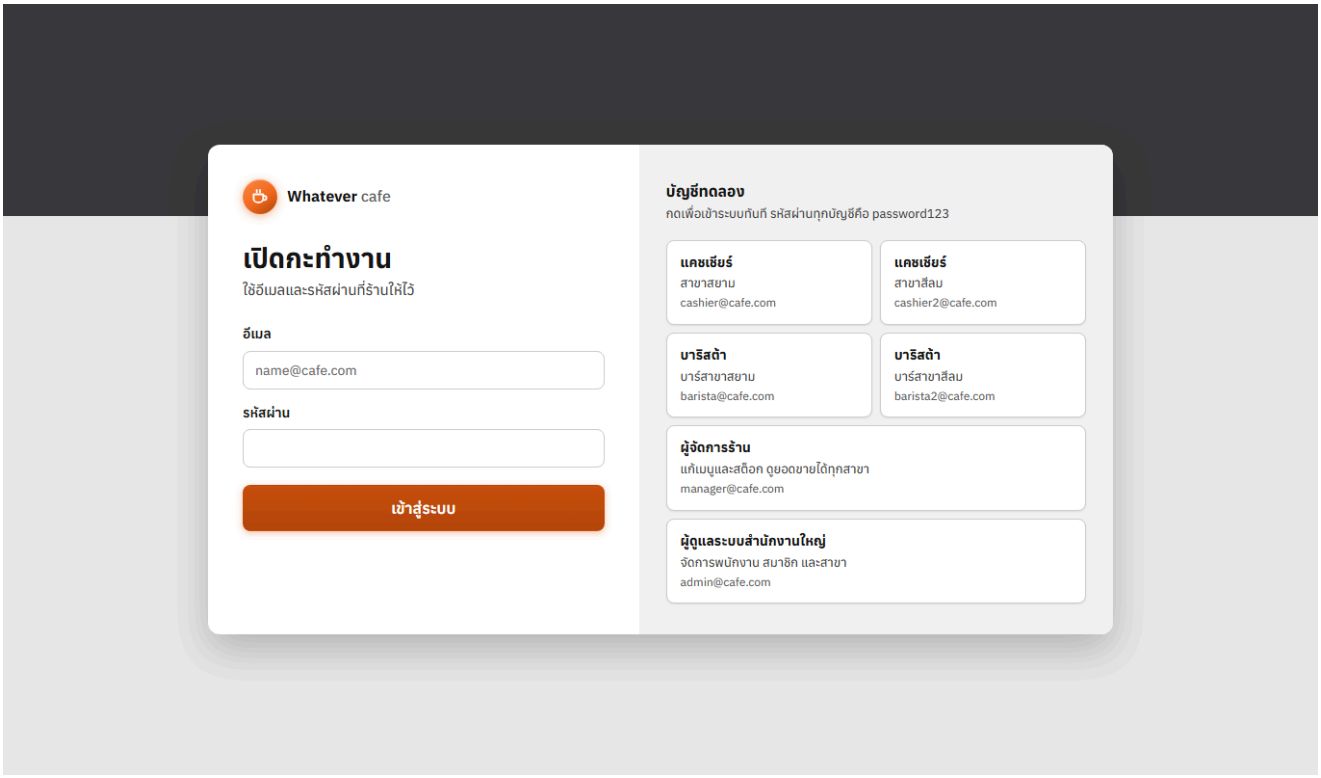
ข้อความแจ้งเมื่อกรอกไม่ครบ เช่น ยังไม่เลือกเมนู ("กรุณาเลือกเมนู"), เบอร์สมาชิกไม่ครบ 10 หลัก, ยังไม่เลือกวิธีชำระเงิน และสต็อกไม่พอ ออกแบบไว้ในเอกสาร wk10 ส่วน Validation และ Feedback

ใบเสร็จแบบ Thermal Slip แสดงเลขที่บิล สาขา แคชเชียร์ ข้อมูลสมาชิก รายการสินค้าพร้อมตัวเลือก ยอดรวม ส่วนลด VAT และยอดสุทธิ รายงานยอดขายแสดงรายได้รวม จำนวนบิล ยอดเฉลี่ยต่อบิล สัดส่วนตามสาขา และ 10 อันดับเมนูขายดี

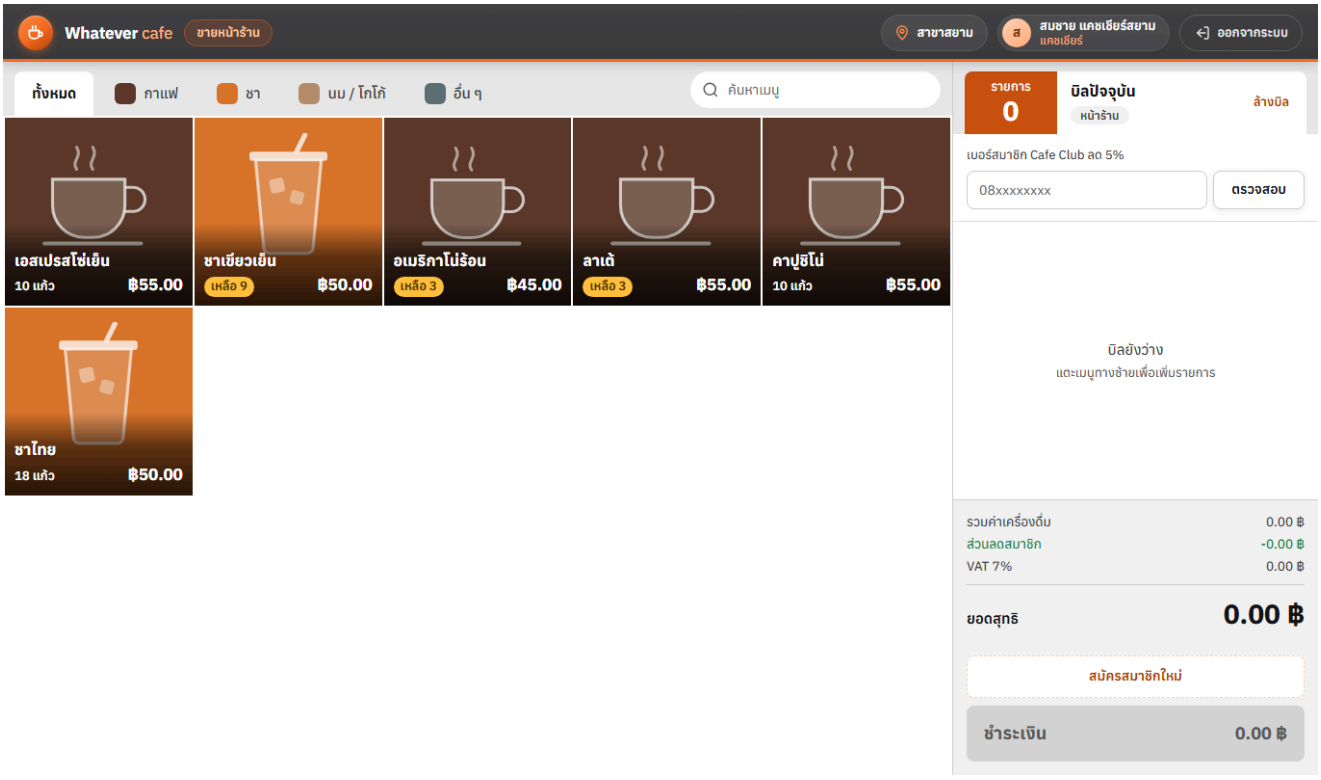
8.3 Wireframe และหน้าจอที่พัฒนาจริง

ไฟล์ wireframe มีรุ่น Wireframe V.1, UI V.1 และ UI V.2 ซึ่งทำก่อนการออกแบบหน้าจอใหม่ ส่วนภาพด้านล่างคือหน้าจอรุ่นปัจจุบันของระบบ (สมาชิกในทีมออกแบบใหม่เมื่อ 19 ตุลาคม 2569 commit 41c29e1)

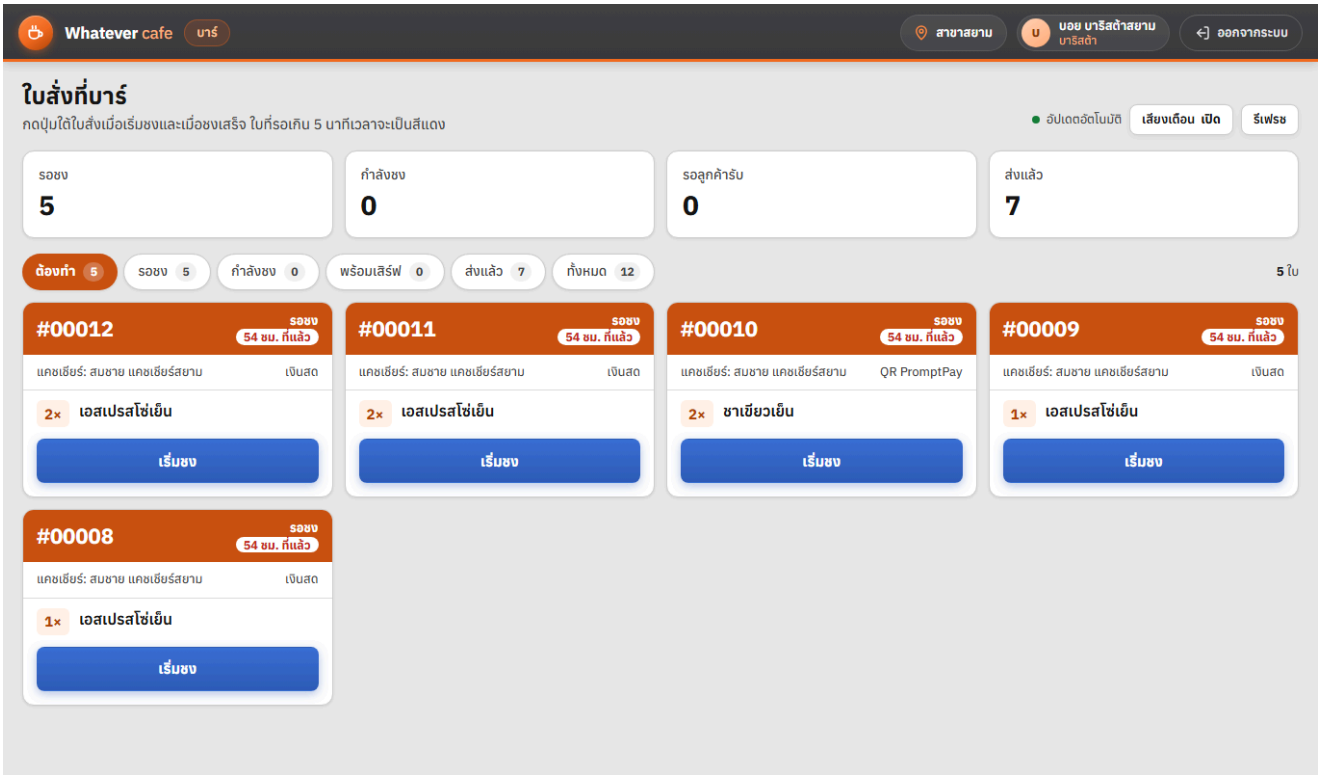
หน้าเข้าสู่ระบบ



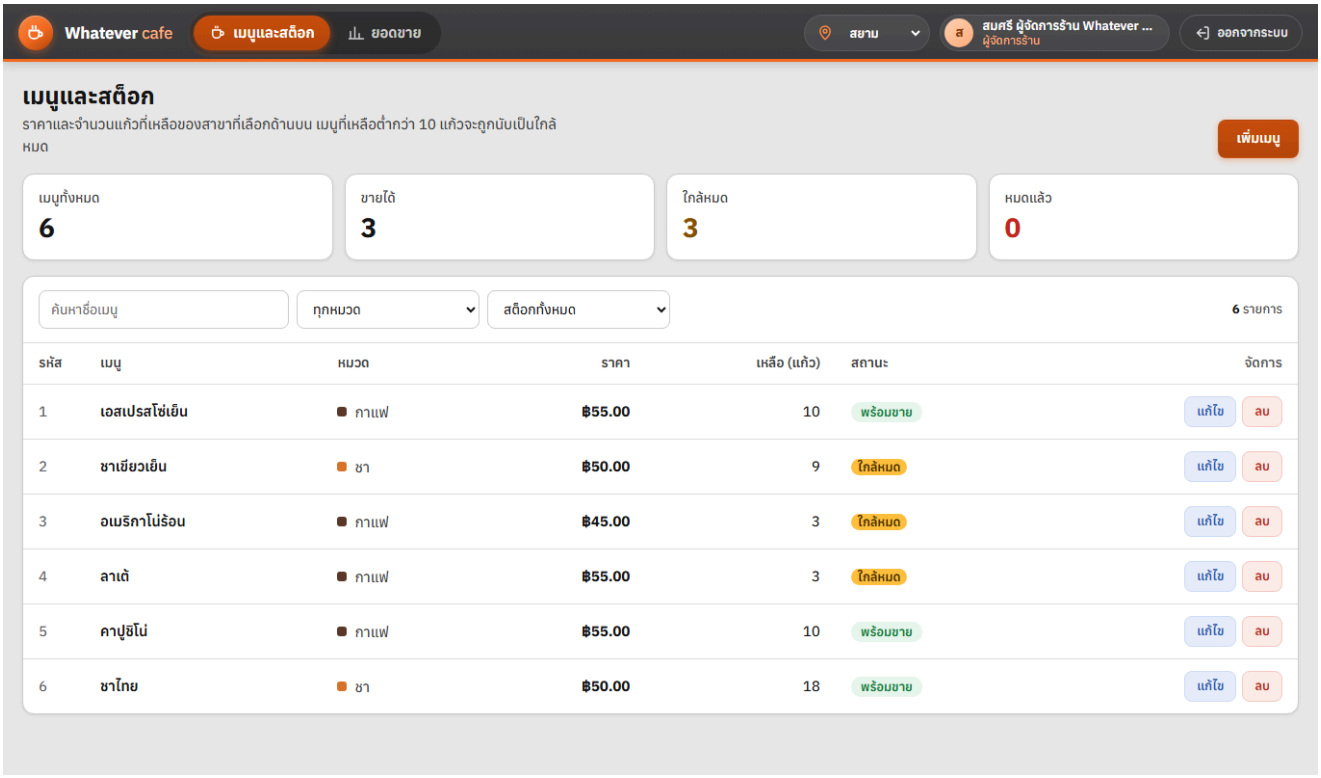
หน้าขายหน้าร้าน (Cashier POS)



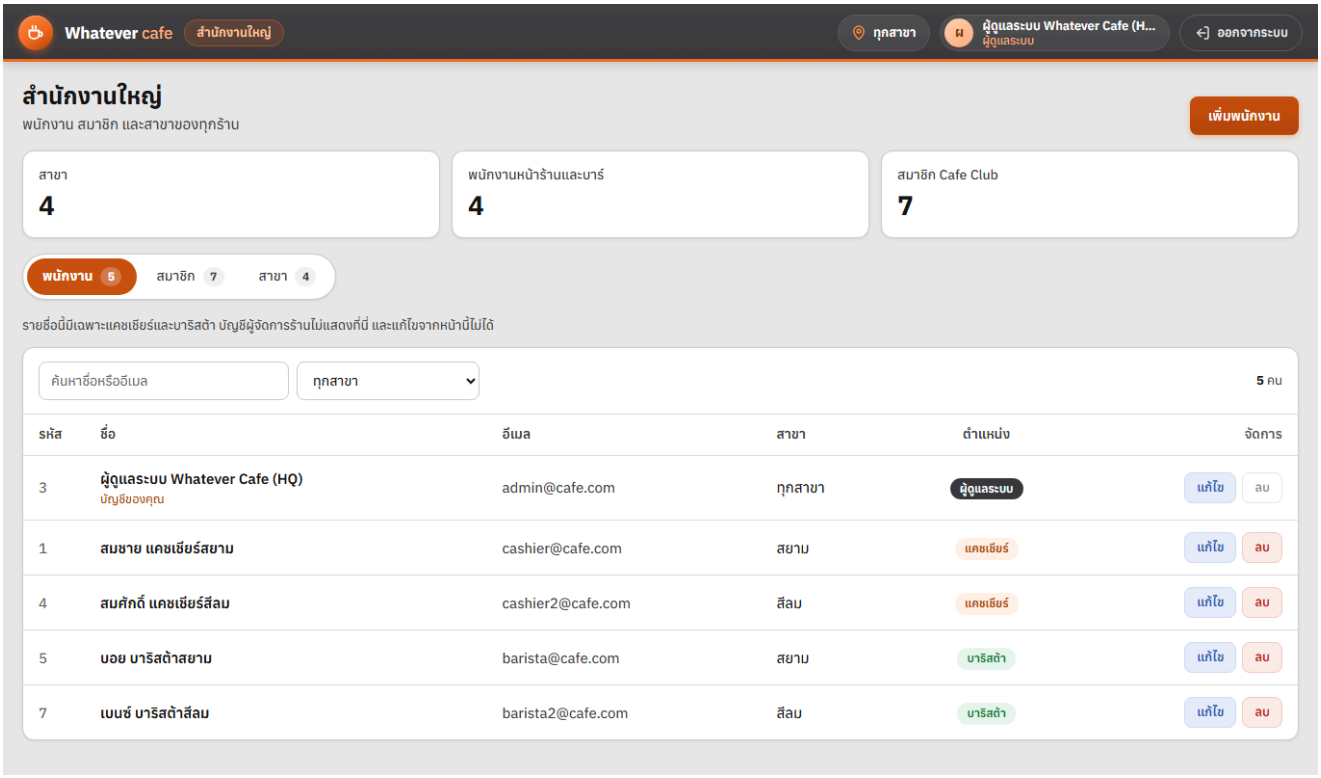
หน้าจอบาริสต้า (KDS)



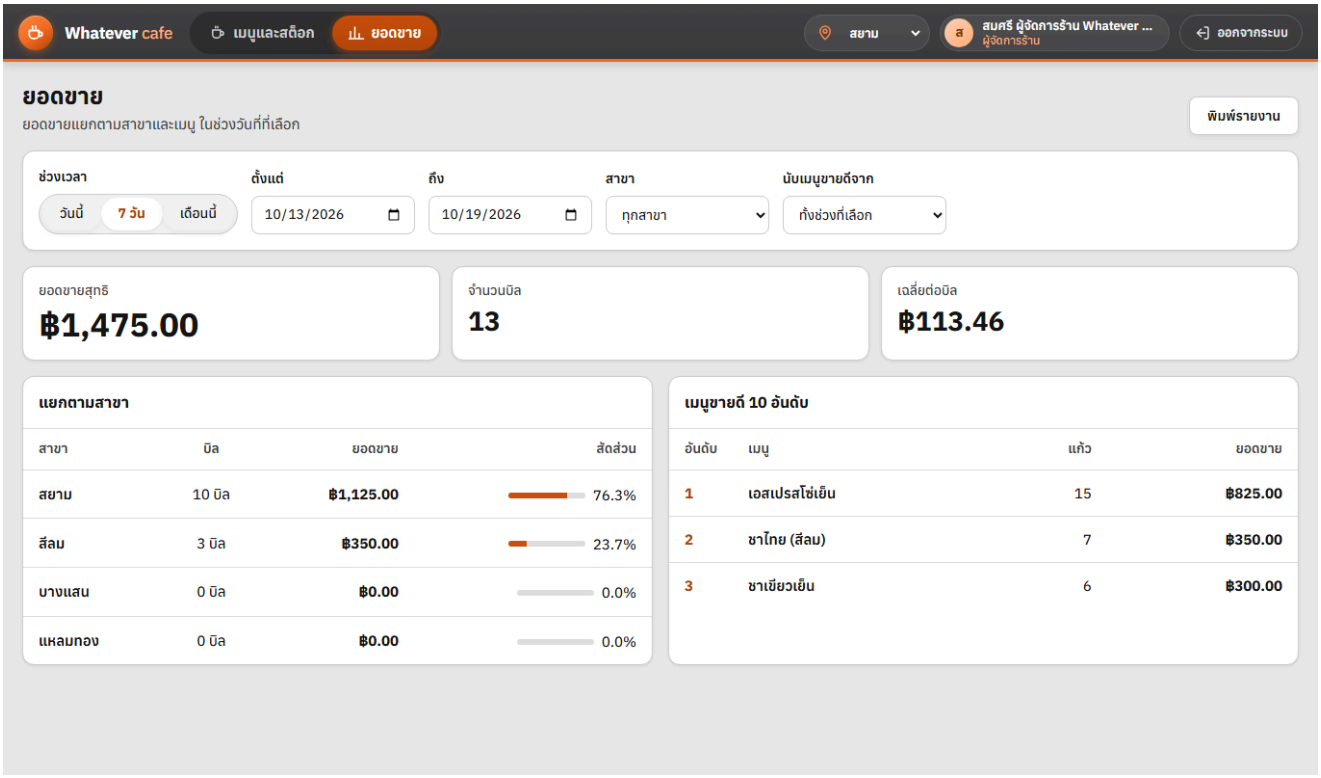
หน้าจัดการเมนูและสต็อก (Manager)



ศูนย์ควบคุมส่วนกลาง (Admin)



หน้ารายงานยอดขาย (Manager)



9. สถาปัตยกรรมระบบ

รายละเอียดฉบับเต็ม (รวม Scenarios, Logical, Process, Development และ Physical View) อยู่ที่ wk11-architecture.md

9.1 รูปแบบสถาปัตยกรรม

ระบบเป็น Monolithic ร่วมกับ Layered Architecture ทำงานเป็น Node.js process เดียว ให้บริการทั้งหน้าเว็บและ API ที่พอร์ต 3000 และเชื่อมต่อ MySQL ผ่าน connection pool (mysql2/promise)

ชั้น	โฟลเดอร์	หน้าที่
Presentation	public/	หน้าจอ UI และการตรวจสิทธิ์ฝั่ง client
Routing	src/routes/	จับคู่ HTTP method และ URL กับ controller และกำหนดสิทธิ์ของแต่ละ endpoint
Controller	src/controllers/	ตรวจข้อมูล คำนวณราคา ส่วนลด VAT และ business logic
Data-access Model	src/models/	คำสั่ง SQL แบบ parameterized query
Configuration	src/config/	การเชื่อมต่อฐานข้อมูล

9.2 Physical View

มี 3 node คือ เบราว์เซอร์ของผู้ใช้ (แท็บเล็ตหรือเดสก์ท็อป) เซิร์ฟเวอร์ Node.js ที่พอร์ต 3000 และ MySQL ซึ่งใน wk11 ระบุพอร์ต 3306 ตามค่าเริ่มต้นของโค้ด (พอร์ตของฐานข้อมูลตั้งผ่าน DB_PORT ใน .env ได้ เช่น เครื่องที่ใช้ทดสอบ wk14 ใช้ 3307 เพราะแมปพอร์ตของ Docker)

9.3 การเปลี่ยนแปลงจาก wk11 เดิมหลังการแก้ไขใน wk14

ประเด็น	ใน wk11	ปัจจุบัน
โฟลเดอร์ใน src/	routes, controllers, models, config	เพิ่ม middleware/ (authMiddleware.js) และ utils/ (password.js สำหรับ hash รหัสผ่านด้วย scrypt)
ขอบเขต transaction	ระบุเป็นข้อจำกัดว่า createOrder รันคำสั่ง SQL แยกกัน	แก้แล้ว ใช้ transaction (ดูหัวข้อ 7.3 และ 12)

diagram ใน wk11 ไม่ได้ปรับ ตามแนวทางของวิชาที่ไม่ให้ย้อนแก้ไฟล์เดิม

---

10. ผลการทดสอบ Usability

รายละเอียดฉบับเต็มอยู่ที่ wk12-usability-testing.md

10.1 วิธีการและข้อจำกัดของการทดสอบ

ผู้ทดสอบคือ AI ไม่ใช่ผู้ใช้ที่เป็นมนุษย์ ทีมใช้ AI (Claude) ลองใช้ระบบจริงที่ http://localhost:3000 ตาม Task ที่กำหนด ในบทบาท Cashier สาขาสยาม เมื่อวันที่ 17 ตุลาคม 2569 จึงไม่มีคำพูดแบบ think-aloud ของมนุษย์ และบันทึกเป็นการสังเกตสิ่งที่ระบบแสดงแทน ทดสอบรอบเดียว ผู้ทดสอบ 1 ราย และหน้าต่างเบราว์เซอร์เตี้ยกว่าจอใช้งานจริง ผลจึงเป็นข้อมูลเบื้องต้น ไม่ใช่ข้อสรุปเชิงสถิติ และทดสอบกับหน้าจอรุ่นเดิม ต่อมาทีมออกแบบหน้าจอใหม่ในวันที่ 19 ตุลาคม 2569 จึงตรวจซ้ำปัญหาทั้ง 3 ข้อกับหน้าจอรุ่นใหม่ (หัวข้อ 10.4)

10.2 Task ที่ใช้

Task	ที่มา	ผล
1 สั่งเครื่องดื่ม 2 แก้ว (แก้วหนึ่งหวานน้อย) ชำระเงินสด จนได้ใบเสร็จ	UC-01 Main Flow, US-01, US-02	ทำสำเร็จ
2 ลูกค้าอยากได้เมนูที่สต็อกหมด จัดการจนจบออเดอร์ได้	UC-01 Alt 2a, US-05	ทำสำเร็จ (ผู้ทดสอบเลือกเมนูอื่นเอง)
3 สมาชิก Cafe Club ได้ส่วนลด แล้วชำระด้วย QR PromptPay	UC-01, US-06, US-02	ทำสำเร็จจนถึงหน้าชำระ QR ส่วนลดสมาชิกถูกต้อง

10.3 ปัญหาที่พบและการดำเนินการ

#	ปัญหา	Severity	การดำเนินการ
1	ปุ่ม "รับชำระเงินเรียบร้อย & ออกใบเสร็จ" ในหน้าต่าง QR กดได้ทันทีโดยไม่มีการยืนยัน	Major	ยังไม่แก้ บันทึกเป็น FR-12 ทีมเห็นว่าการชำระ QR ในต้นแบบเป็นของจำลองที่ไม่เชื่อมเงินจริง และการแก้ต้องเปลี่ยน UI
2	แตะเมนูที่หมดแล้วไม่มีข้อความตอบกลับ (การ์ดมีสีเทาและป้าย "หมด" อยู่แล้ว)	Minor	ยังไม่แก้ บันทึกเป็น FR-11
3	หน้าต่าง QR มีแถบเลื่อนภายในและข้อความ "PromptPay" กลางรูป QR ถูกตัด	Cosmetic	ไม่บังคับแก้

รายการ "ปุ่มชำระเงินต้องเลื่อนหน้า" ถูกตัดออกจากผลสรุป เพราะเกิดจากหน้าต่างทดสอบที่เตี้ยกว่าจอจริง

ข้อสังเกต: แนวทางของวิชากำหนดว่าปัญหาระดับ Major ควรถูกนำไปแก้ไขจริง ปัญหาที่ 1 ยังไม่ได้แก้ตามการตัดสินใจของทีม และระบุเป็นข้อจำกัดในหัวข้อ 13

10.4 ตรวจซ้ำหลังออกแบบหน้าจอใหม่ (19 ตุลาคม 2569)

ชื่อปุ่มบางปุ่มเปลี่ยนในหน้าจอรุ่นใหม่ เช่น "รับชำระเงินเรียบร้อย & ออกใบเสร็จ" เป็น "ได้รับเงินแล้ว ออกใบเสร็จ" ผลการตรวจซ้ำ (AI ผ่านเบราว์เซอร์และการอ่านโค้ด)

#	ปัญหา	ผลกับหน้าจอรุ่นใหม่	สถานะ
1	ปุ่มยืนยันรับเงิน QR กดได้ทันที	ยังพบ กดครั้งเดียวบันทึกออเดอร์ มีเพียงปุ่มยกเลิก	ยังไม่แก้ (FR-12)
2	แตะเมนูที่หมดแล้วไม่มีข้อความตอบกลับ	ยังพบ (ตรวจจากโค้ด ไม่ได้ทดสอบหน้าจอเพราะไม่มีเมนูหมดในข้อมูลทดสอบ)	ยังไม่แก้ (FR-11)
3	หน้าต่าง QR มีแถบเลื่อนและข้อความ "PromptPay" ถูกตัด	แถบเลื่อนหายแล้ว (ตรวจที่ 1280x800 และ 1024x640) แต่ข้อความ "PromptPay" ในกรอบสีน้ำเงินยังถูกตัด	แถบเลื่อนแก้แล้ว ข้อความยังไม่แก้

11. ผลการรวมระบบ (Integration)

ทดสอบ flow เต็ม Order, Payment (บันทึกวิธีชำระ) และ Report ด้วย 10 กรณีทดสอบ (TC-13-01 ถึง TC-13-10) โดยเรียก API จริงของระบบที่รันอยู่ในวันที่ 17 ตุลาคม 2569 (ทดสอบผ่าน HTTP ไม่ได้กดผ่านหน้าจอ)

กลุ่ม	กรณีทดสอบ	ผล
API contract	ตรวจรูปแบบ response ของเมนู	ครั้งแรกไม่ผ่าน (ใช้ snake_case) แก้และทดสอบซ้ำผ่านใน wk14
Flow หลัก	สั่งซื้อ ตัดสต็อก ปรากฏในรายงาน แจ้งเตือนสต็อกต่ำ	ผ่าน ตัวเลขสต็อกและยอดขายถูกต้อง
กรณีผิดปกติ	สต็อกไม่พอ สั่งข้ามสาขา ส่งชนิดข้อมูลผิด ช่วงวันที่ผิด	ผ่าน ระบบปฏิเสธด้วย 400
สิทธิ์	Cashier เรียกรายงาน เรียกโดยไม่แนบ token	ผ่าน ปฏิเสธ

ผลสรุป: 10 กรณี ผ่าน 9 ไม่ผ่าน 1 ในครั้งแรก และผ่านครบหลังแก้ใน wk14

Bug ที่พบ

#	รายละเอียด	สถานะ
B1	response ของเมนูและรายงานเป็น snake_case ไม่ตรงข้อตกลง camelCase	แก้เฉพาะ endpoint เมนูและรายงาน endpoint อื่นยังเป็น snake_case
B2	ข้อความ error ของเมนูข้ามสาขาทำให้เข้าใจว่าสต็อกหมด	แก้แล้ว
B3	บันทึกออเดอร์ เพิ่มรายการ และตัดสต็อกไม่อยู่ใน transaction เดียว	แก้แล้ว
B4	ปัญหา usability จาก wk12	ยังไม่แก้ ตามหัวข้อ 10

สิ่งที่ไม่ได้ทดสอบในรอบนี้: การส่งคิวเข้าหน้าจอบาริสต้า การชำระ QR การเลือกท็อปปิ้ง การสะสมแต้ม และ concurrency (ส่วนของ concurrency ทดสอบเพิ่มใน wk14)

---

12. ผลการตรวจสอบคุณภาพและความปลอดภัย

รายงานฉบับเต็มอยู่ที่ wk14-quality-review.md

12.1 วิธีการ

ทีมไล่ตรวจโค้ดจริงทุก endpoint (33 endpoint) ตาม Code Review Checklist 5 ข้อ คือ การตรวจข้อมูลนำเข้า การป้องกัน IDOR การป้องกัน SQL Injection การจัดการ error และ race condition จากนั้นแก้ความเสี่ยงที่พบและทดสอบซ้ำ **การตรวจ การแก้โค้ด และการทดสอบในหัวข้อนี้ทำโดยใช้ AI (Claude) ช่วย** ส่วนการตัดสินใจว่าจะแก้หรือไม่แก้ประเด็นใดเป็นของทีม

12.2 ความเสี่ยงที่พบและสถานะ

รหัส	ความเสี่ยง	ระดับ	สถานะ
R1	authenticate ไม่บังคับล็อกอิน แต่เป็นตัวป้องกันเดียวของ 17 endpoint ทำให้เข้าถึงข้อมูลได้โดยไม่ล็อกอิน	Critical	แก้แล้ว ใช้ requireAuth และ requireRole
R2	/api/auth/users เปิดให้ทุกคน	High	แก้แล้ว จำกัดเฉพาะ admin
R3	race condition และไม่ atomic ในการตัดสต็อกตอนสร้างออเดอร์	High	แก้แล้ว ใช้ transaction, FOR UPDATE และตัดสต็อกแบบมีเงื่อนไข
R4	เก็บรหัสผ่านเป็น plaintext	High	แก้แล้ว hash ด้วย scrypt อัปเกรดอัตโนมัติเมื่อล็อกอิน
R5	secret ของ token เขียนตายตัวในโค้ดและรับ token ทาง query string	Medium	แก้แล้ว รวมถึงเปลี่ยนค่าใน .env.example เป็น placeholder (เครื่องที่เคยใช้ค่าเดิมควรสร้างค่าสุ่มใหม่)
R6	ตรวจข้อมูลนำเข้าไม่ครบ	Medium	แก้แล้ว
R7	login แยกข้อความ ไม่จำกัดจำนวนครั้ง	Medium	แก้แล้ว ข้อความเดียว ผิดครบ 5 ครั้งล็อก 15 นาที
R8	route ซ้ำ ไม่มี LIMIT สถานะย้อนกลับได้	Low	แก้แล้ว
R9	หน้ารายงานแสดง 0 เพราะตัวแปรไม่ถูกประกาศ	High	แก้แล้ว
R10	login อีเมลที่ไม่มีในระบบได้ 500	Low	แก้แล้ว
R11	order.js error ตอนเพิ่มเมนูลงบิล	Medium	แก้แล้ว
R12	ส่วนลด 0% ถูกบันทึกเป็น 5%	Medium	แก้แล้ว
R13	ไม่เคยบันทึก stock_movement	Medium	แก้แล้ว
R14	dependency มีช่องโหว่ 4 รายการ	High	แก้แล้ว npm audit เหลือ 0
R15	พิมพ์ใบเสร็จได้ 2 หน้า (ไม่มีกฎ @media print)	Low	แก้แล้ว

รหัส	ความเสี่ยง	ระดับ	สถานะ
R16	ข้อความจากฐานข้อมูลถูกใส่ innerHTML โดยไม่ escape ในบางหน้า (พบหลังทีมออกแบบหน้าจอใหม่)	Medium	ยังไม่แก้ ตรวจจากโค้ด

จุดที่เลือกแก้ก่อนคือ R1 (Critical) และ R2 (High) เพราะความรุนแรงสูงสุด แล้วแก้ต่อเนื่องในรอบที่ 2 และ 3 การแก้ R15 และบั๊กฝั่ง frontend (R9, R11, R12) แก้เฉพาะ JavaScript และ CSS สำหรับพิมพ์ โดยไม่เปลี่ยนหน้าตาที่ผู้ใช้เห็น

12.3 ผลทดสอบหลังแก้

ทดสอบซ้ำวันที่ 18 ตุลาคม 2569 โดยเรียก API จริง ชุดทดสอบผ่านครบ 81 รายการ (70 รายการชุดหลักและ 11 รายการชุดเพิ่มเติม) รวมกรณีสั่งพร้อมกัน 6 คำขอสำหรับเมนูเหลือ 1 แก้ว สำเร็จ 1 และปฏิเสธ 5 สต็อกไม่ติดลบ ทดสอบผ่านหน้าจอจริงของทุกบทบาท และ npm audit พบ 0 ช่องโหว่ ผลทดสอบผ่านหน้าจอนี้ทำกับหน้าจอรุ่นเดิมหลังทีมออกแบบหน้าจอใหม่ (19 ตุลาคม 2569) ตรวจซ้ำทุกบทบาทกับหน้าจอรุ่นใหม่แล้ว ผ่านทั้งหมด โดย backend ไม่เปลี่ยน การแก้ R9, R11, R12, R15 และ B1 ยังอยู่ในโค้ดใหม่ และไม่พบข้อผิดพลาดใน console ของเบราว์เซอร์ (หัวข้อ 3.6 ของ wk14-quality-review.md)

หมายเหตุ: ชุดทดสอบบางข้อขึ้นกับข้อมูลตั้งต้นในฐานข้อมูล (เช่น เมนูที่สต็อกเป็น 0) หากสต็อกถูกเติมแล้ว ชุดทดสอบเดิมอาจรายงานว่าไม่ผ่านโดยที่โค้ดไม่ผิด ต้องตั้งสต็อกตามเงื่อนไขก่อนทดสอบ

12.4 ความเสี่ยงและงานที่ยังเหลือ

รายการ	สถานะ	ระดับ
B1: endpoint อื่น (/api/orders/queue, /api/orders, /api/admin/*, /api/branches) ยังตอบ snake_case	แก้บางส่วน ต้องแก้พร้อมกันทั้งสองฝั่ง	Low-Medium
ปัญหา usability: ปุ่มยืนยันรับเงิน QR	ยังไม่แก้ (ต้องเปลี่ยน UI)	ตามหัวข้อ 10
ปัญหา usability: แตะเมนูที่หมดไม่มีข้อความ	ยังไม่แก้ (ต้องเปลี่ยน UI)	Low
บัญชีผู้ใช้ที่ยังไม่เคยล็อกอินยังเก็บรหัสผ่านแบบ plaintext จนกว่าจะล็อกอินครั้งแรก	ควร seed ใหม่ก่อนส่งมอบ	Medium
R16: จุดที่นำข้อความจากฐานข้อมูล (ชื่อเมนู ชื่อสาขา รายละเอียดรายการ) ใส่ innerHTML โดยไม่ escape ใน menu.js, report.js และ order.js (ทีมเพิ่ม escapeHtml แล้วบางส่วนในรอบออกแบบหน้าจอใหม่)	ยังไม่แก้ ตรวจจากโค้ด ยังไม่ได้ทดสอบโจมตีจริง	Medium
ยังไม่ได้ตรวจ HTTPS และ rate limiting ของทั้งระบบ	นอกขอบเขต Checklist	ไม่ระบุ

13. สรุปผลและข้อเสนอแนะ

13.1 สิ่งที่ทำได้

ระบบ POS ทำงานครบ flow ตั้งแต่ล็อกอินตามบทบาท สั่งและปรับแต่งเครื่องดื่ม ชำระเงิน (เงินสดและ QR แบบจำลอง) ออกใบเสร็จ ส่งคิวให้บาริสต้า ตัดสต็อกตามสาขา ระบบสมาชิก และรายงานยอดขาย มีการตรวจคุณภาพและความปลอดภัยของทุก endpoint และแก้ความเสี่ยงสำคัญแล้ว

13.2 บทเรียนที่ได้

- diagram ที่ออกแบบก่อนเขียนโค้ดต้องกลับมาเทียบกับโค้ดจริงเป็นระยะ เพราะโค้ดเปลี่ยนจาก diagram เดิมหลายจุด (ฐานข้อมูล การตรวจสิทธิ์ ธุรกรรม)
- การตรวจโค้ดตาม checklist พบปัญหาที่การทดสอบการใช้งานปกติไม่พบ เช่น endpoint ที่เรียกได้โดยไม่ล็อกอิน และสต็อกติดลบเมื่อสั่งพร้อมกัน
- ผลการทดสอบ usability ที่ได้จาก AI เป็นข้อมูลเบื้องต้น ควรทดสอบกับผู้ใช้จริงก่อนใช้งานจริง

13.3 ข้อจำกัดของระบบและเอกสาร

ข้อจำกัด	หัวข้อที่เกี่ยวข้อง
การชำระเงิน QR เป็นของจำลอง ไม่เชื่อมระบบธนาคารจริง และปุ่มยืนยันกดได้ทันที	10, 12
Class Diagram (wk06) ต่างจากโค้ดปัจจุบัน: ไม่มีคลาส Order และ OrderItem ในโค้ด (มีใน commit a51ca50 ของ wk06 และถูกลบใน 7b4521e) และใช้ตาราง user กับ role แทน Employee, Cashier, Barista	5
ER Diagram และ wk07-schema.sql (wk07) เป็นเวอร์ชันแรก ต่างจาก schema จริงที่มี 8 ตาราง	6
Sequence Diagram (wk09) และ Architecture (wk11) ไม่ได้ปรับให้รวม transaction การตรวจสิทธิ์ และโฟลเดอร์ middleware/, utils/	7, 9
endpoint ของระบบบางส่วนยังตอบ field แบบ snake_case (B1)	11, 12
ยังมีจุดที่ใส่ข้อความจากฐานข้อมูลลง innerHTML โดยไม่ escape (R16)	12
ต้นแบบยังไม่มีการขอความยินยอมสมาชิกและกำหนดระยะเวลาเก็บข้อมูลตาม PDPA	2
หน้าคิวบาริสต้าดึงข้อมูลทุก 3.5 วินาที ยังไม่ถึงเป้าหมาย 2 วินาทีของ NFR-01 และยังไม่ได้วัดเวลาจริง	3, 11
ผลทดสอบ usability มาจากผู้ทดสอบ AI 1 ราย	10
ทดสอบ integration ผ่าน API ไม่ได้ทดสอบผ่านหน้าจอทุก flow	11
ตัวนับการล็อกอินผิดอยู่ในหน่วยความจำของ process เดียว	12
การจำลองหลายสาขาใช้ฐานข้อมูลเดียว (branch_id) ไม่มีการ replicate	9

13.4 แนวทางพัฒนาต่อ

1. วาด ER Diagram, Class Diagram, Sequence Diagram และ Architecture Diagram ใหม่ให้ตรงกับระบบปัจจุบัน และปรับ wk07-schema.sql
2. ให้ทุกเครื่องของทีมสร้าง JWT_SECRET ค่าสุ่มของตนเองใน .env (ค่าเดิมเคยอยู่ใน repository)
3. เพิ่มหน้าต่างยืนยันการรับเงิน QR (FR-12) และข้อความเมื่อแตะเมนูที่หมด (FR-11)
4. แปลง response ของ endpoint ที่เหลือเป็น camelCase พร้อมแก้ frontend ทั้งสองฝั่งพร้อมกัน
5. ทดสอบ usability กับผู้ใช้จริง เพิ่มการขอความยินยอมสมาชิกตาม PDPA แก้จุด XSS ที่เหลือ (R16) และตรวจ HTTPS และ rate limiting
6. เชื่อมต่อผู้ให้บริการชำระเงิน QR จริงเมื่อพร้อม

---

14. ภาคผนวก

14.1 AI Disclosure Form ที่ใช้ตลอดเทอม

สัปดาห์	ไฟล์
wk05	wk05-ai-disclosure.md
wk07	wk07-ai-disclosure.md
wk09	wk09-ai-disclosure.md
wk10	wk10-ai-disclosure.md
wk11	wk11-ai-disclosure.md
wk12	wk12-ai-disclosure.md
wk13	wk13-ai-disclosure.md
wk14	wk14-ai-disclosure.md

14.2 รายการเอกสารรายสัปดาห์

สัปดาห์	เอกสาร
wk01	wk01-team-charter.md
wk02	wk02-feasibility-report.md
wk03	wk03-requirements.md
wk04	wk04-user-stories.md
wk06	wk06-class-diagram.png, wk06-object-diagram.png
wk07	wk07-er-diagram.png, wk07-schema.sql
wk09	wk09-sequence-diagram.png
wk10	wk10-input-output-design.docx, wk10-wireframe.docx, wk10-srs.md
wk11	wk11-architecture.md
wk12	wk12-usability-testing.md
wk14	wk14-quality-review.md

14.3 วิธีรันระบบ

ขั้นตอนติดตั้งและรันอยู่ใน README.md ที่ root ของ repository